Car make and model classification and verification using Residual networks

Tommaso Leoni†, Giulio Bugarella†,

***Abstract*—Accurate recognition and classification of vehicles in the automotive domain is a challenging problem due to the vast number of car makes and models. This task is crucial for applications such as smart city development and traffic monitoring. In this work, we propose a lightweight custom residual neural network trained from scratch that performs car make and model classification and verification with high accuracy. The network is also trained to predict maximum vehicle speed directly from images. A multi-headed architecture enables joint feature extraction with task-specific heads, providing a unified framework for classification, regression, and verification. Cross entropy loss was employed for classification, mean squared error for speed regression, and contrastive loss for verification, where different margin values were tested. Strong verification performance was also achieved on car makes and models unseen during training, demonstrating the adaptability and potential of the proposed network.**

***Index Terms*—Automotive industry, Vehicle recognition, Supervised Learning, Convolutional Neural Networks, Residual Neural Networks.**

I. Introduction

The automotive industry comprises tens of thousands of car models and brands, with new variations continuously emerging. Accurately recognizing and classifying these vehicles is a complex task, as differences between models are often subtle and can easily be overlooked even by experts. This challenge becomes particularly relevant in large scale applications such as traffic monitoring, smart city development, and e-commerce platforms for automotive products.

To address this problem, we propose the use of a custom residual neural network specifically designed for car model and make classification and verification. Residual networks (ResNets) have been proven highly effective in visual recognition tasks by enabling the training of deeper architectures while mitigating vanishing gradient issues. Unlike most existing approaches that rely on transfer learning with pre-trained models, in this work we train our network entirely from scratch. This choice allows us to investigate the true representational power of the architecture in the automotive classification domain, without depending on prior knowledge from unrelated datasets.

The main contributions of this report can be summarized as follows:

- We design and train a custom residual neural network for car make and car moodel classification. The network is evaluated using both cross-entropy and focal loss functions, applied separately in order to highlight their respective advantages and limitations.
- We address a car speed prediction task directly from images, where the model is trained using mean squared error (MSE) as the regression loss.
- We extend the same backbone to a car make and car model verification task, where the objective is to determine whether two images correspond to the same make or model. The network is trained using a contrastive loss function. We test it on both seen and unseen classes during training.

To integrate these tasks, we design a multi-headed architecture where each task-specific head is trained and evaluated independently, while their contributions are weighted according to task relevance.

This report is structured as follows. In Section II, we provide a brief overview of related literature. Section III gives a high-level description of our approach. The data processing and model architecture are detailed in Sections IV and V, respectively. The results are presented and discussed in Section VI; Section VII concludes our findings.

II. Related Work

The foundation of residual networks was laid by He et al. [1], [2].

In the domain of fine-grained car recognition (make/model classification), there are several works that leverage CNNs and large datasets (e.g. the CompCars dataset) and often use pre-trained networks + fine-tuning. For example [3].

What distinguishes the current work is that training is performed from scratch (i.e. no fine-tuning of a pre-trained model), with standard initialization in the form of Glorot. Unlike Hu et al., who build on networks already trained on large generic datasets, our methodology aims to explore how well a custom residual architecture can learn all tasks (make classification and verification, model classification and verification, speed prediction) starting from zero.

III. Processing Pipeline

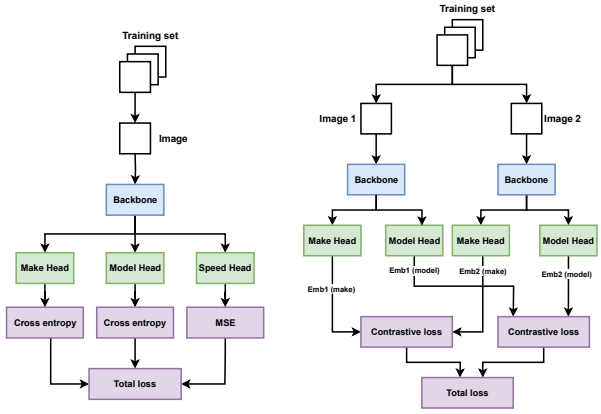
The proposed processing pipeline is organized around a custom residual network serving as a shared backbone for multiple vision tasks. Input images are first passed through this backbone to extract high level feature representations, which are then routed to task specific heads (make, model and speed) depending on the learning objective. The classification heads are trained with cross-entropy or focal loss functions, while the regression head is optimized using mean squared

†Department of Information Engineering, University of Padova,
email: {tommaso.leoni, giulio.bugarella}
@studenti.unipd.it

error (MSE). These tasks are trained jointly in a multi-task learning setup.

For the verification tasks, we generate positive pairs (images of cars belonging to the same class) and negative pairs (images from different classes) from the dataset, as described in Sec. IV. We then pass them through the same backbone with dedicated embedding layers, producing normalized feature vectors for both car make and car model. Training is performed with a contrastive loss, encouraging embeddings of images from the same class to cluster together while pushing apart those from different classes. The backbone and the two heads are trained simultaneously: in the same way as for the classification task, the total loss is given by a weighted sum of the contrastive losses.

A schematic overview of the pipeline is illustrated in Fig. 1a Fig. 1b, highlighting the shared backbone and the separation between multi-task classification/regression and verification branches.



(a) Classification pipeline (b) Verification pipeline

Fig. 1: Comparison between classification and verification pipelines

IV. Signals and Features

The CompCars dataset contains car images from web-nature and surveillance-nature scenarios [4]. This project utilizes the web-nature subset of the CompCars dataset, which consists of images obtained from car forums, public websites, and search engines. The images are organized in a hierarchical structure based on car make, model, and year, and feature various attributes and viewpoints. A total of 163 car makes with 1716 car models are represented.

The first step we have taken is to filter out makes and models for which the speed is not available. This gives us a total of 116 makes and 806 models with complete information. For the classification task, augmentation is then applied to all training images to improve generalization. The augmentation pipeline includes:

- Random horizontal flipping with probability 0.5.
- Random rotation within ± 15 degrees.
- Color jittering (brightness, contrast, saturation, hue).
- Random resized cropping to 224×224 with variable scale and aspect ratio.
- Conversion to tensor and normalization using mean $[0.485, 0.456, 0.406]$ and standard deviation $[0.229, 0.224, 0.225]$ (as in ImageNet preprocessing).

The fully processed dataset consists of 81286 samples, of which 5000 were kept out for the final test, while the remaining were split into training (80%) and validation (20%). After preprocessing the resulting image tensors were directly used as input to our custom ResNet. For the verification task image pairs were generated separately for each subset. Specifically, we constructed 50,000 pairs from the training set, 10,000 pairs from the validation set, and 10,000 pairs from the test set. Each set contained 40% of same make same model pairs, 20% of same make different model, 40% of different make. An additional set of 50000 pairs with the same characteristics was generated from the remaining images (those discarded because without complete attributes) in order to test the verification network also on classes unseen during training. Each pair is annotated with two binary labels: one for car make and one for car model. A label value of 1 indicates that both images belong to the same class, while a value of 0 denotes different classes. This setup enables the network to learn discriminative embeddings at both make and model levels. No data augmentation was applied to the images for the verification task, since an extremely high number of pairs could have been potentially generated starting from the dataset, while the resizing and normalization was the same as for the classification.

V. Learning Framework

The learning framework adopted in this work is based on a custom ResNet convolutional neural network presented in Fig. 2. The backbone comprises 34 layers (including BatchNorms), which allow for a lightweight architecture.

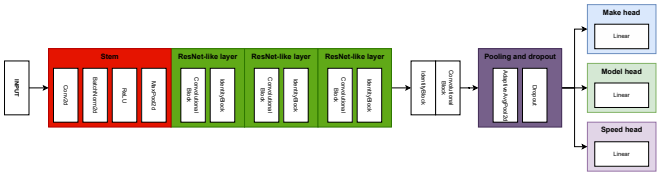


Fig. 2: Network schema for classification. For verification only the heads are modified, while the backbone is the same.

In identity blocks, the input and output have the same dimensions, so the shortcut adds the input directly to the transformed features. Convolutional blocks instead reduce the spatial resolution (stride = 2) and use a 1x1 convolution in the shortcut to match dimensions before addition.
The backbone was trained from scratch, with weights initialized using the Glorot method.

For the first phase, the network was trained for a multi-task problem combining car make and model classification and maximum velocity regression. Cross-entropy loss was used for

classification and was then compared with experiments using focal loss to mitigate class imbalance. In the latter case, the alpha parameter was set inversely proportional to the square of the frequency of each class. Maximum velocity regression was trained using the mean squared error (MSE) loss. After empirical tuning, the total loss used for training the multi-task model was a weighted combination:

$$\mathcal{L}_{\text{total}} = 1 \cdot \mathcal{L}_{\text{make}} + 0.8 \cdot \mathcal{L}_{\text{model}} + 0.0015 \cdot \mathcal{L}_{\text{MSE}}. \quad (1)$$

The second phase focused on car make and model verification. The weights of the backbone were initialized from the previously trained classification model to leverage the learned feature representation. The same random seed as the one of the loaded checkpoint was used. Two separate embedding heads were added, obtaining as output 2 embeddings for each image, the first one representing the make and second one the model (embedding dimension = 128). Given the two make embeddings $\mathbf{e}_{1,\text{make}}$ and $\mathbf{e}_{2,\text{make}}$, and a binary label $y_{\text{make}} \in \{0, 1\}$ indicating whether the two images belong to the same class, the Euclidean distance is computed as

$$d_{\text{make}} = \|\mathbf{e}_{1,\text{make}} - \mathbf{e}_{2,\text{make}}\|_2. \quad (2)$$

The make loss is then formulated as

$$\mathcal{L}_{\text{make}} = y_{\text{make}} \cdot d_{\text{make}}^2 + (1 - y_{\text{make}}) \cdot \max(0, m - d_{\text{make}})^2, \quad (3)$$

where m is a margin hyperparameter, for which two values were tested: $m = 1.5$ and $m = 2.0$. Loss for the model was computed in the same way. The total loss for verification was a weighted sum of the individual losses:

$$\mathcal{L}_{\text{total}} = 0.4 \cdot \mathcal{L}_{\text{make}} + 0.6 \cdot \mathcal{L}_{\text{model}} \quad (4)$$

The backbone was fine-tuned during verification training, as freezing everything beside the final 3 layers resulted in much slower convergence.
For both tasks an Adam optimizer (weight decay = 10^{-4}) was employed with an initial learning rate of 10^{-3}; for the classification training it was scheduled to decrease to 10^{-6} over 50 epochs (through Cosine Annealing), while for verification it was set to decrease to 10^{-5} in 25 epochs. A batch size of 128 was used, as it provided the most stable and rapid training progress.

VI. Results

We measured the classification and verification performances in terms of accuracy, i.e., the fraction of correctly classified samples, and the maximum velocity regression in terms of root mean square error (RMSE).

A. Classification and regression

Since for the classification task the training process has been repeated with 3 different seeds, we report the mean and standard deviation for each value.

	Train acc.	**Val. acc.**	**Test (Top 1)**	**Test (Top 5)**
make	0.967 $\pm 0,00$	0.958 $\pm 0,002$	0.878 ± 0.011	0.968 $\pm 0,003$
model	0.907 $\pm 0,00$	0.914 $\pm 0,003$	0.802 $\pm 0,005$	0.949 $\pm 0,001$

TABLE 1: Classification results with CE loss

Train MSE	**Val. MSE**	**Test MSE**	**Test RMSE (km/h)**
$163,90$ $\pm 1,99$	$165,03$ $\pm 13,02$	$258,27$ $\pm 24,44$	16,07 $\pm 4,94$

TABLE 2: Max speed regression results with CE loss. For test both MSE and RMSE are reported in order to have a comparison with training and validation results.

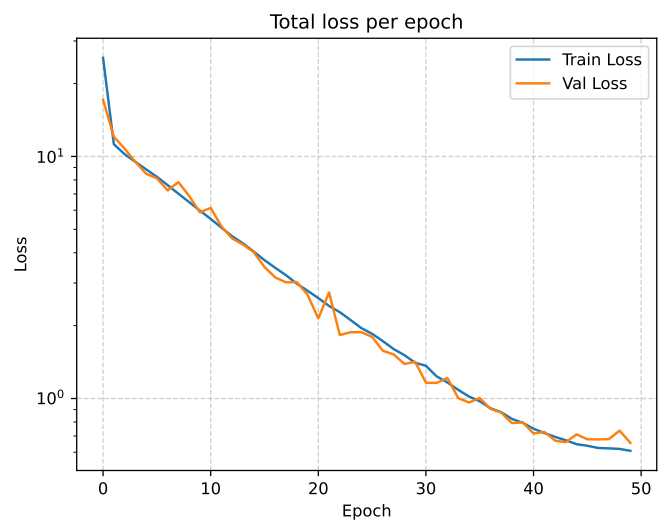


Fig. 3: Train and validation loss for the training with CE loss.

	Train acc.	**Val. acc.**	**Test (Top 1)**	**Test (Top 5)**
make	$0,380$ $\pm 0,006$	$0,392$ $\pm 0,005$	$0,369$ $\pm 0,009$	$0,598$ $\pm 0,006$
model	$0,355$ $\pm 0,007$	$0,379$ $\pm 0,006$	$0,323$ $\pm 0,006$	$0,530$ $0,006$

TABLE 3: Classification results with focal loss

Train MSE	**Val. MSE**	**Test MSE**	**Test RMSE (km/h)**
$265,40$ $\pm 4,53$	$270,73$ $\pm 28,60$	$374,16$ $\pm 33,05$	$19,34$ $\pm 5,75$

TABLE 4: Max speed regression results with focal loss. For test both MSE and RMSE are reported in order to have a comparison with training and validation results.

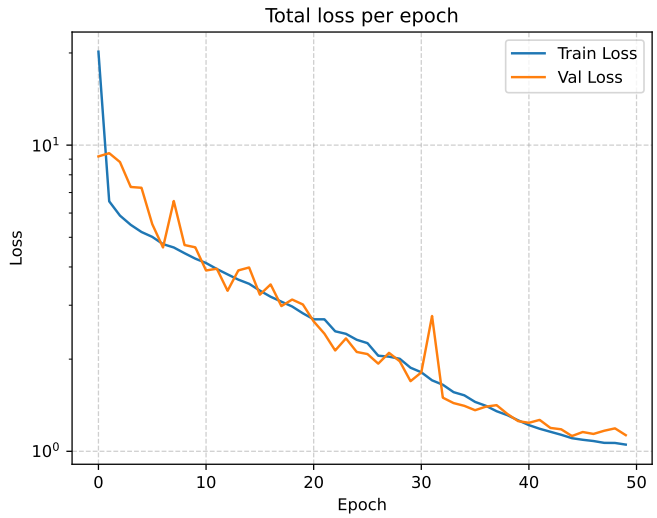


Fig. 4: Train and validation loss for the training with focal loss.

From Tab.1 and Tab.2 and Tab.3 and Tab.4 we can see that the training using cross entropy leads to a vastly superior result in all the tasks. Training with focal loss was not able to converge even on training set.
Examples of classification (probabilities are obtained through SoftMax output) are shown in Fig.5. Majority of wrong predictions are caused by extremely similar models or makes, difficult to distinguish even by human eye.

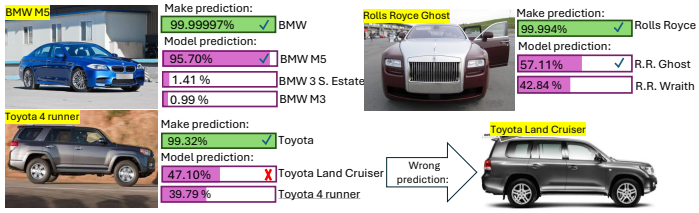


Fig. 5: Examples of classification results. BMW image is from CompCars dataset, the other images are imported from web.

B. Verification

Results for verification task are reported in terms of accuracy in Tab.5 and Tab.6. We show results for $m = 1.5$ and $m = 2.0$ (margin value used in contrastive loss) in order to compare the two approaches.

	Train acc.	**Val acc.**	**Test acc.**
make	0.893	0.866	0.826
model	0.919	0.887	0.825

TABLE 5: Accuracy with $m = 1.5$ (seed 60)

	Train acc.	**Val acc.**	**Test acc.**
make	0.911	0.893	0.852
model	0.937	0.909	0.857

TABLE 6: Accuracy with $m = 2.0$ (seed 60)

For this task, accuracies were computed using a threshold (one for make and one for model), chosen as the value that maximized the accuracy on the validation set:

$$\text{pred} = 1 \quad \text{if} \quad d < t, \quad \text{otherwise} \quad \text{pred} = 1 \tag{5}$$

Depending on the value of m, the following thresholds were found as optimal:

$$m = 1.5 : \quad \begin{cases} t_{make} = 0.808 \\ t_{model} = 0.895 \end{cases}$$

$$m = 2.0 : \quad \begin{cases} t_{make} = 0.800 \\ t_{model} = 0.846 \end{cases}$$

An example of how the threshold determines the prediction is shown in Fig. 6.

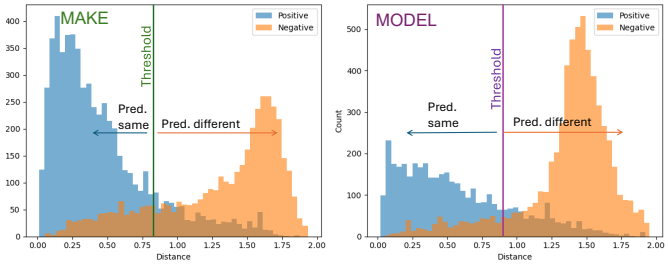


Fig. 6: Histogram of distances between embeddings on test set, $m = 2.0$

More detailed prediction values for each m value on test set are reported in Tab.7 and Tab.8, where true label (1;1) means pairs of same make, same model images; (1;0) same make, different model; (0;0) different make.

	True labels		
	(1;1)	(1;0)	(0;0)
pred. same ($m = 1.5$)	0.919	0.805	0.251
pred. different ($m = 1.5$)	0.081	0.195	0.749
pred. same ($m = 2.0$)	0.885	0.770	0.160
pred. different ($m = 2.0$)	0.115	0.230	0.840

TABLE 7: MAKE verification 2x3 confusion matrix

	True labels		
	(1;1)	(1;0)	(0;0)
pred. same ($m = 1.5$)	0.819	0.300	0.090
pred. different ($m = 1.5$)	0.181	0.700	0.910
pred. same ($m = 2.0$)	0.765	0.177	0.048
pred. different ($m = 2.0$)	0.235	0.823	0.952

TABLE 8: MODEL verification 2x3 confusion matrix

Beside a better accuracy in general, the contrastive loss with $m = 2.0$ provided better results in recognizing pairs of images from different classes. We believe this is a consequence of the greater distance between negative pair embeddings that such value of m achieves (Tab.9). On the other hand, the network showed greater accuracy (3-5% increase) on same class recognition when trained with $m = 1.5$

	$m = 1.5$	$m = 2.0$
Make	1.19	1.33
Model	1.31	1.40

TABLE 9: Mean distance between negative embeddings after training for both values of the margin parameter.

The verification network was also tested on a set of car images from makes and models not included in the training data (as described in Sec. IV). These models had never been seen by the network during training, including during the classification phase. On such set of images, the network showed great adaptability (only the network trained with $m = 2$ was tested on this set):

$$\text{Make verification accuracy}: \quad 0.740 \tag{6}$$

$$\text{Model verification accuracy}: \quad 0.762 \tag{7}$$

The ROC plot on such set is reported in Fig.7. To compute it, we used the inverse of the distance between embeddings as the score, so that more similar pairs (positives) correspond to higher score values.

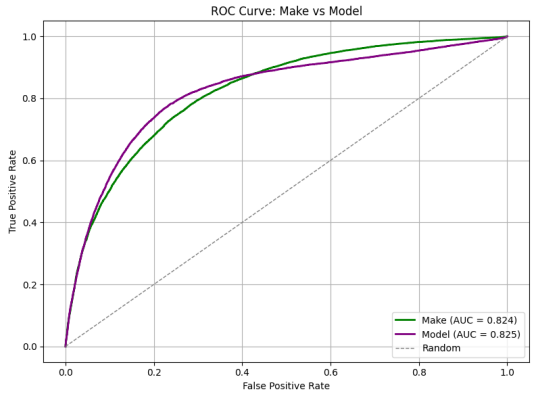


Fig. 7: ROC on set of images from unseen classes.

VII. Concluding Remarks

In this work, we presented a lightweight residual neural network trained from scratch for fine-grained vehicle recognition and verification. The model was able to jointly address classification, regression, and verification tasks within a unified framework, with some results largely above 80% accuracy. The exploration of different loss functions (cross entropy, focal loss, mean squared error, and contrastive loss) provided insights into their respective strengths across tasks. Finally, the verification network achieved robust performance not only on seen classes but also when generalizing to unseen makes and models, highlighting its adaptability. Future work could focus on training the network on larger datasets, and implement a different head for each viewpoint. An optimization of the architecture for real time deployment could also be useful for traffic monitoring applications.

Given the results previously seen an actual possible application for the presented architecture could be in a car factory. A car manufacturer could place an imaging system at the end of the production line in order to generate a dataset comparable with the CompCars dataset, which would be used to train network to recognize the different kinds of models produced. A dataset of images taken from the same view point would likely lead to greater accuracy on similar models recognition. Once trained the network could be shared with the car dealers in order to verify car shipments.

During this project, we learned how a lightweight neural network, trainable on a portable PC, can achieve impressive results in object recognition tasks, even without pretrained weights. We also learned how a single network can be trained for multiple tasks simultaneously, optimizing computational resources. One challenge we encountered was the reproducibility of results across different devices, which limited us to training and testing on the same device.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *European conference on computer vision*, pp. 630–645, Springer, 2016.

[3] Q. Hu, H. Wang, T. Li, and C. Shen, "Deep cnns with spatially weighted pooling for fine-grained car recognition," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 11, pp. 3147–3156, 2017.

[4] L. Yang, P. Luo, C. Change Loy, and X. Tang, "A large-scale car dataset for fine-grained categorization and verification," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3973–3981, 2015.